4. `garde_fou.py` expliqué

Nutri-Arbitre — fiche de révision · étape 4 sur 5 du trajet des données (download → clean → tools → **garde_fou** → agent) · **brique 3**

En une phrase : avant d'afficher la réponse de Qwen, ce fichier vérifie que **chaque nombre** qu'elle contient vient d'un résultat d'outil (ou de la question). Si un seul nombre n'a pas d'origine, la réponse est **bloquée** et remplacée par un message de refus.
Brique 3 (une règle imposée par le code) : la règle « aucun chiffre non calculé » est appliquée par le *code*, pas seulement demandée dans le *prompt*. Un test essaie de l'enfreindre et se fait bloquer.
Deuxième règle (ajoutée le 2 novembre) : un outil n'a pas le droit de filtrer ou de découper en groupes selon le bio ou le Nutri-Score si la question n'en parle pas.
Où elles sont appliquées : dans `agent.py`. La règle des chiffres sur la réponse finale (fonction `repondre`), la règle des filtres au moment de chaque appel d'outil (fonction `executer_outil`).

Pourquoi le code et pas seulement le prompt ?

> Le prompt de `agent.py` dit déjà à Qwen : « n'écris jamais un nombre que tu n'as pas reçu d'un outil ». Mais **un prompt est une demande, pas une obligation** : le modèle peut l'ignorer, surtout un petit modèle. On l'a vu le 2 novembre : Qwen a calculé lui-même un chiffre malgré la consigne.
>
> Le garde-fou, lui, est du code Python : il s'exécute à chaque fois, et le modèle ne peut pas le contourner. **Le prompt dit la règle, le code la fait respecter.**

La méthode en 3 étapes

1. **Trouver les nombres de la réponse** (`extraire_nombres`).
2. **Faire la liste des nombres autorisés** : ceux des résultats d'outils, ceux de la question, et les constantes 100 et 95 (`nombres_autorises`).
3. **Vérifier chaque nombre de la réponse** : est-il un arrondi correct d'un nombre autorisé ? (`nombre_justifie`). Si un seul ne l'est pas, on bloque (`verifier`, `filtrer_reponse`).

Étape 1 : trouver les nombres (lignes 16 à 34)

```
21 MOTIF_NOMBRE = re.compile(r"-?\d+(?:[.,]\d+)?")
```

C'est une **expression régulière** : un « modèle » de texte à rechercher. On la lit morceau par morceau :

Morceau	Veut dire	Exemple
`-?`	un signe moins, facultatif (le `?` = « 0 ou 1 fois »)	`-` dans « -5,42 »

`\d+`	un ou plusieurs chiffres (`\d` = un chiffre, `+` = « au moins une fois »)	« 13 »
`(?:[.,]\d+)?`	facultatif : un point ou une virgule, puis des chiffres	« ,2 » dans « 13,2 »

`findall` renvoie tous les morceaux de texte qui correspondent. Exemple réel : « 100 g contiennent 13,2 ± 0,7 g… un bol de 40 g… 5,3 g… 14,0 g » → `['100', '13,2', '0,7', '40', '5,3', '14,0']`. `en_float` transforme ensuite « 13,2 » en 13.2, car Python ne comprend que le point.

Étape 2 : les nombres autorisés (lignes 23 à 48)

```
47      texte = json.dumps(objet, ensure_ascii=False, default=str)
48      return [en_float(n) for n in extraire_nombres(texte)]
```

Les résultats d'outils sont des dictionnaires, parfois avec des listes dedans. **L'astuce :** on les transforme en texte JSON avec `json.dumps`, puis on y cherche les nombres avec *la même* expression régulière qu'à l'étape 1. Exemple : `{"n": 543, "moyenne": 13.17}` devient le texte `'{"n": 543, "moyenne": 13.17}'`, qui donne `[543.0, 13.17]`.

- On ajoute les nombres de **la question** : si l'utilisateur parle d'un bol de 40 g, Qwen a le droit de répéter « 40 g ».
- On ajoute les **constantes** 100 (« pour 100 g ») et 95 (« marge à 95 % »).
- Ligne 79 : on ajoute aussi chaque nombre **sans son signe** (`abs`). Un écart de –5,42 peut s'écrire « 5,42 g de moins ».

Étape 3 : chaque nombre est-il justifié ? (lignes 51 à 71)

Qwen arrondit : il écrit « 13,2 » pour 13,17. Il faut accepter un arrondi *correct*, mais refuser un arrondi trop large. La règle : **l'écart permis est une demi-unité du dernier chiffre écrit**.

```
65      tolerance = 0.5 * 10 ** (-decimales)
```

Qwen écrit	Décimales	Écart permis	Avec 18,37 calculé
18	0	0,5	écart 0,37 → ✅ accepté
18,4	1	0,05	écart 0,03 → ✅ accepté
19	0	0,5	écart 0,63 → ❌ bloqué
18,5	1	0,05	écart 0,13 → ❌ bloqué

C'est exactement la règle de l'arrondi qu'on apprend à l'école : 18,37 s'arrondit à 18 ou à 18,4, jamais à 19. Ligne 66, on ajoute une marge minuscule (0,000000001), car un ordinateur calcule parfois 0,1 + 0,2 = 0,30000000000000004 : sans elle, un arrondi juste pourrait être refusé par erreur.

La boucle des lignes 68 à 71 compare le nombre écrit à *chaque* nombre autorisé : dès qu'un nombre colle, `return True`. Si aucun ne colle, `return False`.

Le verdict : `verifier` et `filtrer_reponse` (lignes 74 à 91)

- **`verifier`** fait les étapes 2 et 3, et renvoie deux choses : la réponse est-elle autorisée (`True` / `False`), et la liste des nombres fautifs. Par exemple `(False, ['5,3'])`.
- **`filtrer_reponse`** est la barrière : si tout est justifié, elle renvoie la réponse telle quelle ; sinon, elle renvoie le message « Réponse bloquée… » avec les nombres fautifs.
- Une réponse **sans aucun chiffre** est toujours acceptée (« Je ne peux pas répondre » est honnête).

> **Ce qui s'est vraiment passé (2 novembre 2026)** : deux questions qui poussent Qwen à calculer lui-même.
>
> | Question | Ce qu'a fait Qwen | Garde-fou |
> |---|---|---|
> | « Combien de sucre dans un bol de 40 g de céréales bio ? » | Appelle `decrire` → 13,17 g pour 100 g. Calcule lui-même 13,17 × 0,4 = **5,3 g**. Le calcul est **juste**. | ❌ bloqué : « 5,3 » `traces/trace_20261102_113046.json` |
> | « Combien de calories dans une portion de 30 g de céréales ? » | Filtre à tort les seules céréales **non bio**, puis calcule lui-même 416,04 × 0,3 et écrit « 124,9 kcal exactement, calculé à partir des données » (le bon résultat est 124,8). | ❌ bloqué : « 125, 124,9 » `traces/trace_20261102_113111.json` |
>
> **Pourquoi bloquer un calcul juste ?** Le garde-fou ne sait pas vérifier un calcul fait par le modèle. La question 2 montre que le même modèle peut se tromper en affirmant que c'est « exact ». Il vaut mieux bloquer un calcul juste que laisser passer un calcul faux. La solution propre serait d'ajouter un outil « pour une portion » : le calcul serait alors fait par le code.

Deuxième règle : pas de filtre dont la question ne parle pas (lignes 94 à 112)

Le problème : à la question « combien de calories dans une portion de 30 g de céréales ? », Qwen a filtré les seules céréales *non bio*. Un nouvel essai, il a découpé bio / non bio. La question portait sur *toutes* les céréales : sa réponse était fausse, et la règle des chiffres ne pouvait pas le voir, car ce n'est pas un chiffre.

```
97  PARLE_DE_BIO = re.compile(r"\b(bio|bios|biologique|biologiques|organic|ab)\b",
re.IGNORECASE)
104     utilise_bio = arguments.get("bio") is not None or arguments.get("groupe") ==
"bio"
105     if utilise_bio and not PARLE_DE_BIO.search(question):
106         return ("la question ne parle pas de bio : ne filtre pas ...")
```

- **`PARLE_DE_BIO`** est une expression régulière qui cherche les mots bio, biologique, organic, AB… `\b` veut dire « début ou fin de mot » : « bio » est trouvé dans « céréales bio », mais pas dans « biologie ». `re.IGNORECASE` : majuscules et minuscules sont traitées pareil.

- `utilise_bio` vaut True si l'appel filtre sur le bio (`bio=True` ou `False`) **ou** découpe en groupes bio / non bio (`groupe="bio"`).
- Si l'appel utilise le bio **et** que la question n'en parle pas, la fonction renvoie un message d'erreur. Même chose pour le Nutri-Score.

Appliquée au moment de l'appel, pas à la fin. Dans `agent.py`, `executer_outil` appelle `verifier_filtres` *avant* de lancer le calcul (lignes 43 à 45). Si l'appel est refusé, le calcul n'est jamais fait, et Qwen reçoit le message d'erreur au lieu d'un résultat. Il peut alors **corriger son appel** et réessayer, au lieu que l'utilisateur reçoive seulement un refus.

> **Vrai essai après l'ajout de la règle (2 novembre 2026)**, même question sur les calories :
>
> 1. Qwen appelle `decrire(energie_kcal, bio=False)` → refusé : « la question ne parle pas de bio… ».
> 2. Qwen **corrige** et appelle `decrire(energie_kcal)` sur toutes les céréales → 409,45 kcal, n = 953.
> 3. Il calcule ensuite lui-même la portion de 30 g → la règle des chiffres bloque « 122,84 ».
>
> Les deux règles ont fonctionné l'une après l'autre. `traces/trace_20261102_113921.json`

Les limites (à dire soi-même)

- **Il vérifie l'origine des chiffres, pas le sens de la phrase.** Le 30 octobre, Qwen a répondu « Non » avec des chiffres qui disaient « Oui » : la réponse est passée.
- **Il ne vérifie qu'une partie du choix de l'outil.** La deuxième règle empêche un filtre bio ou Nutri-Score *en trop*. Mais elle ne voit pas un mauvais nutriment (glucides au lieu de sucres), ni un filtre qui *manque*. Et elle se fie à une liste de mots : une question tournée autrement (« les céréales labellisées… ») serait mal comprise.
- **Qwen ne répond pas deux fois pareil.** À la même question, il a d'abord filtré les non bio, puis comparé bio / non bio, puis pris toutes les céréales. C'est pour ça que la brique 9 lance chaque question plusieurs fois.
- **Une coïncidence peut passer.** Un chiffre inventé qui se trouve être égal à un nombre autorisé (un `n`, un minimum…) est accepté.
- **Les nombres écrits en lettres** (« cinq grammes ») ne sont pas reconnus par l'expression régulière.
- **Les noms de champs sont lus aussi** : « marge_95 » ajoute 95 aux nombres autorisés. On l'a rendu explicite en mettant 95 dans les constantes.

Les tests qui essaient d'enfreindre la règle

`tests/test_garde_fou.py` simule des réponses du modèle, sans appeler Qwen : un chiffre inventé (« 12,5 g ») est bloqué ; une réponse de mémoire, sans outil, est bloquée ; un arrondi trop large (« 19 » pour 18,37) est bloqué ; le vrai calcul de Qwen (« 5,3 ») est bloqué ; un arrondi correct, une réponse sans

chiffre et un nombre de la question sont acceptés. **Lancer :** `python -m pytest tests/test_garde_fou.py -v`

Questions d'examinateur

> 1. Le prompt demande déjà de ne pas inventer de chiffres. Pourquoi ajouter un garde-fou en code ?
> 2. Que fait l'expression régulière `-?\d+(?:[.,]\d+)?` ? Donne un exemple.
> 3. D'où viennent les nombres autorisés ?
> 4. Qwen écrit « 18 » alors que l'outil a calculé 18,37. Est-ce accepté ? Et « 19 » ? Pourquoi ?
> 5. Pourquoi transforme-t-on les résultats d'outils en texte JSON ?
> 6. Le calcul de Qwen pour le bol de 40 g était juste. Pourquoi la réponse a-t-elle été bloquée quand même ?
> 7. Cite une erreur que le garde-fou ne peut pas détecter.
> 8. Pourquoi autorise-t-on la valeur sans signe (`abs`) ?
> 9. Que vérifie la deuxième règle, et pourquoi est-elle appliquée au moment de l'appel d'outil plutôt qu'à la fin ?
>
> Essaie d'y répondre avec tes mots avant de lire le corrigé à la page suivante.

Corrigé

1. **Le code plutôt que le prompt** : un prompt est une demande que le modèle peut ignorer, surtout un petit modèle. Le garde-fou est du code qui s'exécute à chaque réponse : le modèle ne peut pas le contourner. On l'a vu le 2 novembre : malgré le prompt, Qwen a calculé lui-même « 5,3 g », et c'est le code qui l'a bloqué.
2. **L'expression régulière** reconnaît un nombre dans un texte : un signe moins facultatif, des chiffres, puis éventuellement une virgule ou un point suivi de chiffres. Dans « 13,2 ± 0,7 g », elle trouve « 13,2 » et « 0,7 ».
3. **Les nombres autorisés** : tous les nombres des résultats d'outils, ceux de la question de l'utilisateur, les constantes 100 et 95, et chacun de ces nombres sans son signe.
4. **« 18 » oui, « 19 » non** : écrit sans décimale, un nombre peut s'écarter d'au plus 0,5. 18,37 est à 0,37 de 18 (accepté) et à 0,63 de 19 (bloqué). C'est la règle de l'arrondi : 18,37 s'arrondit à 18, pas à 19.
5. **Le texte JSON** : les résultats sont des dictionnaires et des listes emboîtés. En les transformant en texte, on peut y chercher les nombres avec la même expression régulière que pour la réponse, sans parcourir la structure à la main.
6. **Un calcul juste bloqué** : le garde-fou ne sait pas si un calcul du modèle est juste. Il ne vérifie qu'une chose : ce nombre vient-il d'un outil ? 5,3 n'en vient pas, donc bloqué. Et la question 2 montre que le modèle peut se tromper en affirmant que c'est « exact » : on ne peut pas lui faire confiance au cas par cas.
7. **Une erreur non détectée** : au choix, une phrase qui contredit ses propres chiffres (« Non » au lieu de « Oui ») ; un mauvais filtre (non bio au lieu de toutes les céréales) ; un chiffre inventé qui tombe par hasard sur un nombre autorisé ; un nombre écrit en lettres.
8. **La valeur sans signe** : les outils renvoient un écart bio − non bio négatif (−5,42) quand les bio en ont moins. En français, on écrit « 5,42 g de moins », sans le signe moins. Sans `abs`, cette phrase correcte serait bloquée.
9. **La deuxième règle** vérifie qu'un outil ne filtre pas, et ne découpe pas en groupes, selon le bio ou le Nutri-Score quand la question n'en parle pas. On l'applique au moment de l'appel : le calcul filtré n'est jamais fait, et Qwen reçoit un message d'erreur qui lui permet de corriger son appel. C'est ce qui s'est passé le 2 novembre : il a refait l'appel sur toutes les céréales.